

Ahmedabad Institute of Technology  
Department of Computer Engineering

# **Food Order Analysis Using Data Science**

Mini Project Documentation  
Python for Data Science (BE05000231)  
5th Semester -- CE-C

**Submitted by:**

[Student Name 1] -- [Enrollment No.]

[Student Name 2] -- [Enrollment No.]

**Guided by:**

[Faculty Guide Name]

Academic Year: [20XX -- 20XX]

# CERTIFICATE

This is to certify that the mini project entitled "**Food Order Analysis Using Data Science**" has been carried out by **[Student Name 1] ([Enrollment No.]** and **[Student Name 2] ([Enrollment No.]** under my guidance in partial fulfilment of the requirements for the subject **Python for Data Science (BE05000231)**, 5th Semester, CE-C, at **Ahmedabad Institute of Technology**, during the academic year [20XX-20XX].

---

Faculty Guide

---

Head of Department

## ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our faculty guide, **[Faculty Guide Name]**, for their valuable guidance and continuous support throughout the development of this mini project. We are also thankful to the Department of Computer Engineering, Ahmedabad Institute of Technology, for providing the resources and environment necessary to complete this project. Finally, we thank our peers and family for their encouragement.

## ABSTRACT

Understanding customer ordering behavior is essential for online food platforms to optimize operations, marketing, and customer satisfaction. This project analyzes 1,898 real-world food orders from the FoodHub dataset to uncover patterns in order value, cuisine popularity, restaurant performance, and customer rating behavior. The workflow covers data cleaning, exploratory data analysis, feature engineering, and a supervised classification model to predict whether an order receives a high customer rating. Two classification models -- Logistic Regression and Random Forest -- were trained and compared. The analysis found that American and Italian cuisines dominate order volume, weekend orders substantially outnumber weekday orders, and a striking 39% of orders are never rated by the customer at all. Order value and restaurant popularity emerged as the strongest available predictors of a high rating, though neither the Logistic Regression nor Random Forest model was able to outperform a simple majority-class baseline -- indicating that rating behavior depends on factors not captured in this dataset, such as food quality or delivery experience. These insights are directly useful for a food delivery platform seeking to increase rating engagement and understand cuisine- and restaurant-level order patterns. This project is scoped strictly to food ORDER analysis; delivery logistics (preparation and delivery time) were intentionally excluded.

# TABLE OF CONTENTS

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>CERTIFICATE</b>                                         | <b>2</b>  |
| <b>ACKNOWLEDGEMENT</b>                                     | <b>3</b>  |
| <b>ABSTRACT</b>                                            | <b>4</b>  |
| <b>TABLE OF CONTENTS</b>                                   | <b>5</b>  |
| <b>LIST OF ABBREVIATIONS</b>                               | <b>7</b>  |
| <b>Chapter 1: Introduction</b>                             | <b>8</b>  |
| 1.1 Background . . . . .                                   | 8         |
| 1.2 Problem Statement . . . . .                            | 8         |
| 1.3 Objectives . . . . .                                   | 8         |
| 1.4 Scope of the Project . . . . .                         | 8         |
| 1.5 Applications . . . . .                                 | 8         |
| <b>Chapter 2: Literature Survey</b>                        | <b>9</b>  |
| <b>Chapter 3: Requirement Analysis</b>                     | <b>10</b> |
| 3.1 Hardware Requirements . . . . .                        | 10        |
| 3.2 Software Requirements . . . . .                        | 10        |
| 3.3 Python Libraries Used . . . . .                        | 10        |
| <b>Chapter 4: Dataset Description</b>                      | <b>11</b> |
| 4.1 Dataset Source . . . . .                               | 11        |
| 4.2 Dataset Information . . . . .                          | 11        |
| 4.3 Attribute Description . . . . .                        | 11        |
| 4.4 Data Quality Analysis . . . . .                        | 11        |
| <b>Chapter 5: System Design</b>                            | <b>12</b> |
| 5.1 System Architecture . . . . .                          | 12        |
| 5.2 Flowchart . . . . .                                    | 12        |
| 5.3 Module Diagram . . . . .                               | 13        |
| 5.4 Workflow Description . . . . .                         | 14        |
| <b>Chapter 6: Project Methodology</b>                      | <b>15</b> |
| <b>Chapter 7: Project Implementation</b>                   | <b>16</b> |
| 7.1 Sample Code -- Feature Engineering (excerpt) . . . . . | 16        |
| <b>Chapter 8: Project Screenshots and Results</b>          | <b>17</b> |
| 8.1 Exploratory Data Analysis . . . . .                    | 17        |
| 8.2 Model Evaluation . . . . .                             | 19        |

|                                                    |           |
|----------------------------------------------------|-----------|
| <b>Chapter 9: Testing and Validation</b>           | <b>20</b> |
| 9.1 Model Validation Results . . . . .             | 20        |
| <b>Chapter 10: Results and Discussion</b>          | <b>21</b> |
| 10.1 Feature Importance . . . . .                  | 21        |
| 10.2 Prediction Accuracy vs. Baseline . . . . .    | 21        |
| <b>Chapter 11: Conclusion</b>                      | <b>22</b> |
| <b>Chapter 12: Future Scope</b>                    | <b>23</b> |
| <b>References</b>                                  | <b>24</b> |
| <b>Appendix</b>                                    | <b>25</b> |
| A.1 Source Code . . . . .                          | 25        |
| Phase 1: Data Collection & Understanding . . . . . | 25        |
| Phase 2: Data Cleaning & Preprocessing . . . . .   | 27        |
| Phase 3: Exploratory Data Analysis . . . . .       | 29        |
| Phase 4: Feature Engineering . . . . .             | 31        |
| Phase 5: Model Building . . . . .                  | 33        |
| Phase 6: Model Evaluation . . . . .                | 35        |
| Phase 7: Results & Conclusion . . . . .            | 37        |
| A.2 Dataset Sample . . . . .                       | 39        |
| A.3 Output Reports . . . . .                       | 40        |

# LIST OF ABBREVIATIONS

| Abbreviation | Full Form                                        |
|--------------|--------------------------------------------------|
| EDA          | Exploratory Data Analysis                        |
| ML           | Machine Learning                                 |
| F1           | F1 Score (harmonic mean of precision and recall) |
| CSV          | Comma-Separated Values                           |
| IQR          | Interquartile Range                              |
| API          | Application Programming Interface                |

# Chapter 1: Introduction

## 1.1 Background

Online food ordering platforms process a high volume of orders across many restaurants and cuisines every day. Understanding the patterns behind these orders -- what customers order, how much they spend, when they order, and whether they are satisfied -- is critical for platform operators to make informed business decisions. Data science techniques make it possible to systematically analyze historical order data to uncover these patterns.

## 1.2 Problem Statement

Food ordering platforms accumulate large amounts of order data, but without structured analysis it is difficult to know which cuisines and restaurants drive the most orders, how order value varies across categories, and what factors are associated with customers leaving a high rating. This project addresses that gap through exploratory and predictive analysis of order-level data.

## 1.3 Objectives

- To collect and clean a real-world food order dataset.
- To perform exploratory data analysis on order value, cuisine, restaurant, and timing patterns.
- To engineer meaningful features describing each order.
- To build and compare classification models predicting whether an order receives a high customer rating.
- To derive actionable business insights from the analysis.

## 1.4 Scope of the Project

This project covers the complete data science pipeline -- from raw order data to a trained classification model -- applied to food order characteristics: order value, cuisine, restaurant, day type, and customer rating. Delivery logistics such as food preparation time and delivery time are explicitly excluded from the analysis, as the project is scoped strictly to food ORDER analysis rather than delivery performance.

## 1.5 Applications

- Identifying top-performing cuisines and restaurants for promotional focus.
- Understanding order value trends across cuisines and day types.
- Improving customer rating engagement (since a large share of orders go unrated).
- Supporting menu and pricing strategy based on order value patterns.



## Chapter 2: Literature Survey

Prior studies on e-commerce and food-ordering platforms commonly use exploratory data analysis to understand purchasing patterns -- order frequency, average order value, and category popularity -- as a foundation for business strategy. Classification models such as Logistic Regression and Random Forest are frequently used in customer satisfaction and churn-prediction studies to identify which factors are associated with positive outcomes. A recurring finding in this literature is that satisfaction and rating behavior are often only weakly explained by transactional attributes (price, category, timing) alone, and are more strongly influenced by product/service quality factors that are not always captured in transactional datasets -- a pattern consistent with the findings of this project.

# Chapter 3: Requirement Analysis

## 3.1 Hardware Requirements

| Component        | Minimum Requirement       |
|------------------|---------------------------|
| Processor        | Dual-core 2 GHz or higher |
| RAM              | 4 GB (8 GB recommended)   |
| Storage          | 500 MB free space         |
| Operating System | Windows / Linux / macOS   |

## 3.2 Software Requirements

| Software               | Version      |
|------------------------|--------------|
| Python                 | 3.9 or above |
| Jupyter Notebook / IDE | Latest       |
| pip (package manager)  | Latest       |

## 3.3 Python Libraries Used

| Library      | Purpose                                                     |
|--------------|-------------------------------------------------------------|
| pandas       | Data loading, cleaning, and manipulation                    |
| numpy        | Numerical computations                                      |
| matplotlib   | Data visualization (base plotting)                          |
| seaborn      | Statistical data visualization                              |
| scikit-learn | Classification models, train-test split, evaluation metrics |
| joblib       | Saving and loading trained models                           |

## Chapter 4: Dataset Description

### 4.1 Dataset Source

The dataset used is the **"FoodHub Data"** dataset, publicly available on Kaggle: <https://www.kaggle.com/datasets/tasnimniger/foodhub-data>

### 4.2 Dataset Information

The dataset contains **1,898 records** and **9 original columns**, each representing one food order placed through the FoodHub platform, covering 1,200 unique customers and 178 unique restaurants.

### 4.3 Attribute Description

| Attribute              | Description                                                                  |
|------------------------|------------------------------------------------------------------------------|
| order_id               | Unique identifier for each order                                             |
| customer_id            | Unique identifier for each customer                                          |
| restaurant_name        | Name of the restaurant the order was placed from                             |
| cuisine_type           | Type of cuisine offered by the restaurant                                    |
| cost_of_the_order      | Order value in dollars (used as a proxy for revenue)                         |
| day_of_the_week        | Whether the order was placed on a Weekday or Weekend                         |
| rating                 | Customer rating (3, 4, 5, or 'Not given')                                    |
| food_preparation_time* | Time to prepare the food (minutes) -- excluded from analysis (out of scope)  |
| delivery_time*         | Time to deliver the order (minutes) -- excluded from analysis (out of scope) |

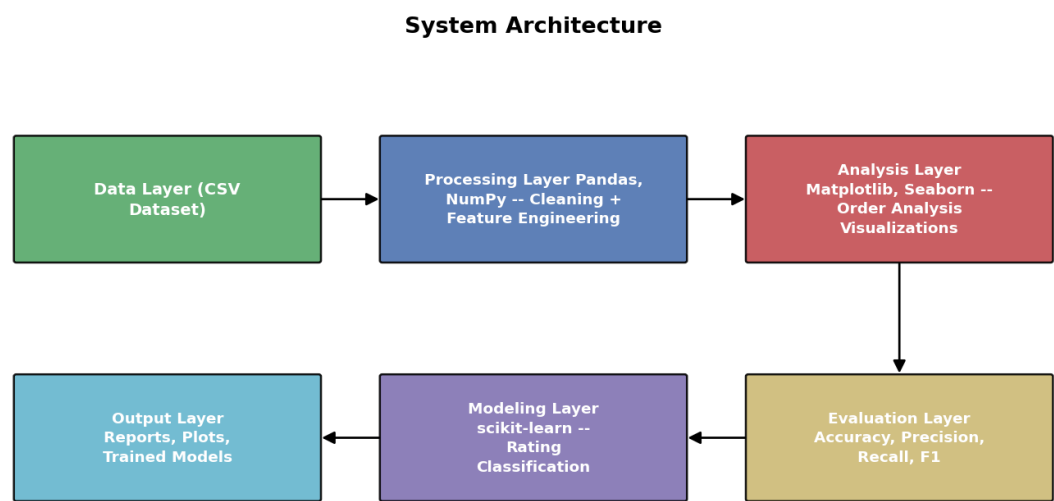
*\* food\_preparation\_time and delivery\_time exist in the raw dataset but were dropped during cleaning (Phase 2), since this project is scoped strictly to food order analysis and not delivery performance.*

### 4.4 Data Quality Analysis

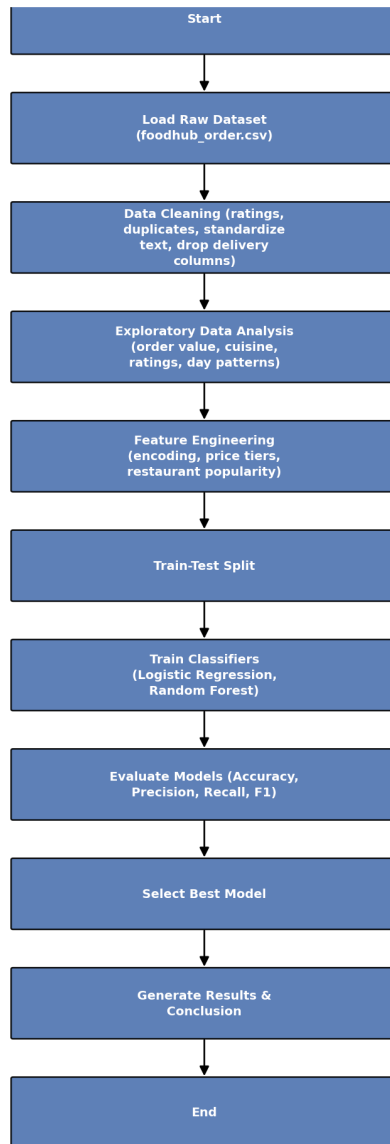
Initial inspection (Phase 1) found no missing (NaN) values and no duplicate order\_ids. The rating column contains a 'Not given' category for orders the customer chose not to rate (736 of 1,898 orders, about 38.8%) -- this was treated as a valid response category rather than missing data. No statistical outliers were found in order value using the IQR method.

# Chapter 5: System Design

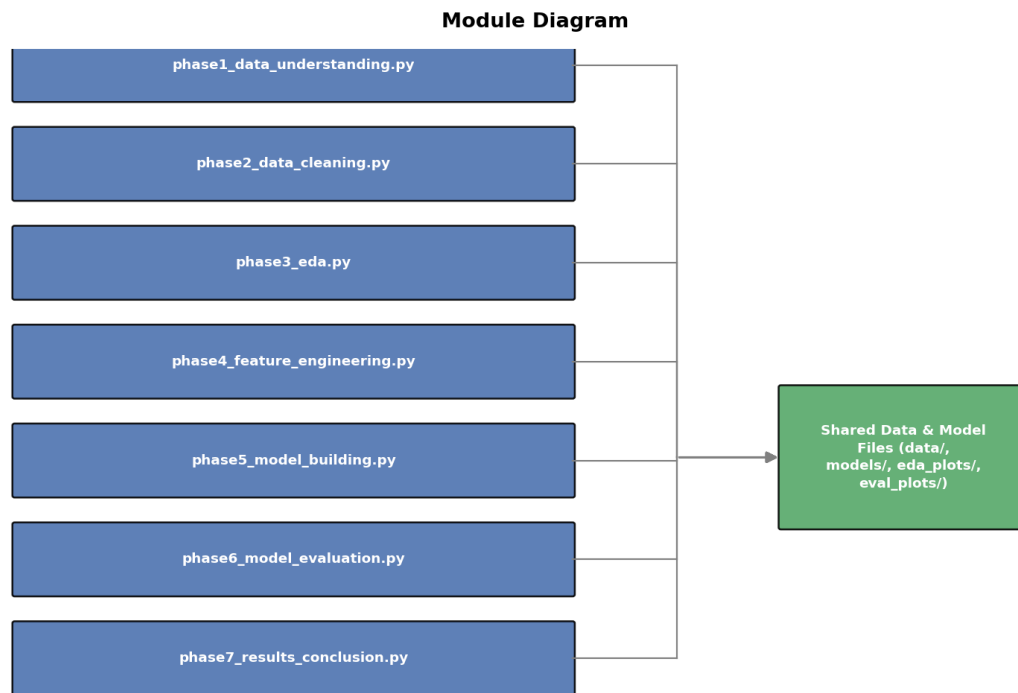
## 5.1 System Architecture



## 5.2 Flowchart



### 5.3 Module Diagram



## 5.4 Workflow Description

The project is implemented as seven sequential Python modules (phase1 through phase7), each reading the output of the previous stage and writing its own output to disk. Every module uses functions with exception handling so that a missing file or malformed value produces a clear error message instead of a crash.

## Chapter 6: Project Methodology

- 1. Data Collection:** Real-world order dataset obtained from Kaggle.
- 2. Data Cleaning:** Duplicate removal, text standardization, rating category handling, outlier flagging on order value, and removal of out-of-scope delivery columns.
- 3. Exploratory Data Analysis:** Visualizations of order value, cuisine popularity, restaurant performance, day-type patterns, and rating distribution.
- 4. Feature Engineering:** One-hot encoding of top cuisines, restaurant popularity (order count), price tier buckets, and a weekend flag. A binary 'high\_rating' target (4 or 5 stars) was created for classification.
- 5. Model Building:** Stratified 80/20 train-test split, feature scaling, training of Logistic Regression and Random Forest classifiers (with hyperparameter tuning via GridSearchCV).
- 6. Model Evaluation:** Accuracy, Precision, Recall, F1 score, and confusion matrices, compared against a majority-class baseline.
- 7. Results & Conclusion:** Consolidation of findings into actionable insights.

## Chapter 7: Project Implementation

All code was implemented in Python using pandas, NumPy, matplotlib, seaborn, and scikit-learn, organized into seven modular scripts corresponding to the seven project phases. Every script uses functions for each logical step, with file I/O and data operations wrapped in try/except blocks. The full source code is provided in the Appendix.

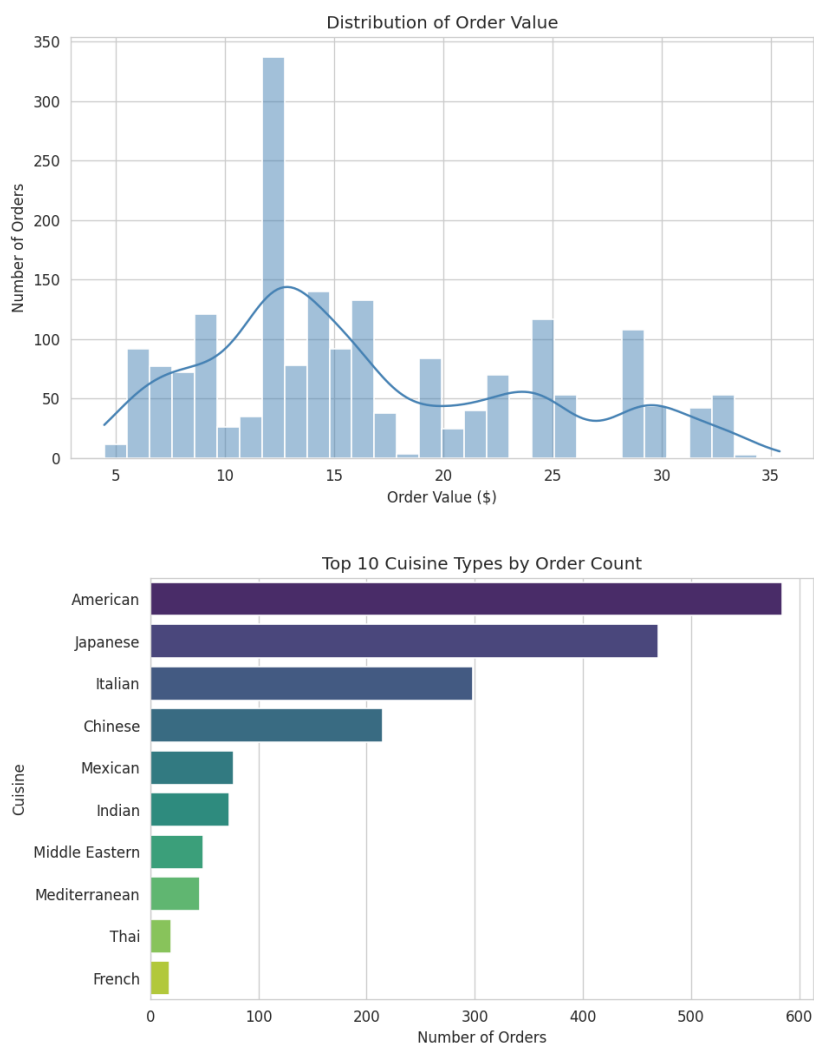
### 7.1 Sample Code -- Feature Engineering (excerpt)

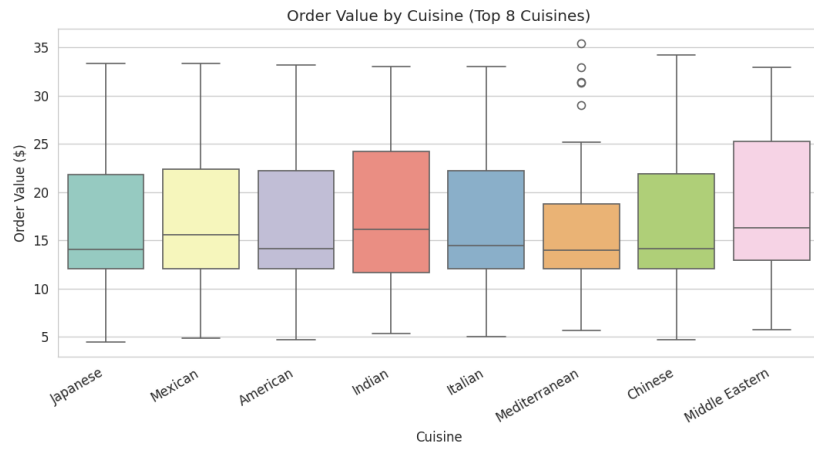
```
def add_restaurant_popularity(df):
    """Number of orders each restaurant has received -- a proxy for popularity/trust."""
    try:
        popularity = df.groupby("restaurant_name")["order_id"].transform("count")
        df["restaurant_order_count"] = popularity
    except KeyError:
        print("Warning: could not compute restaurant popularity.")
    return df
```



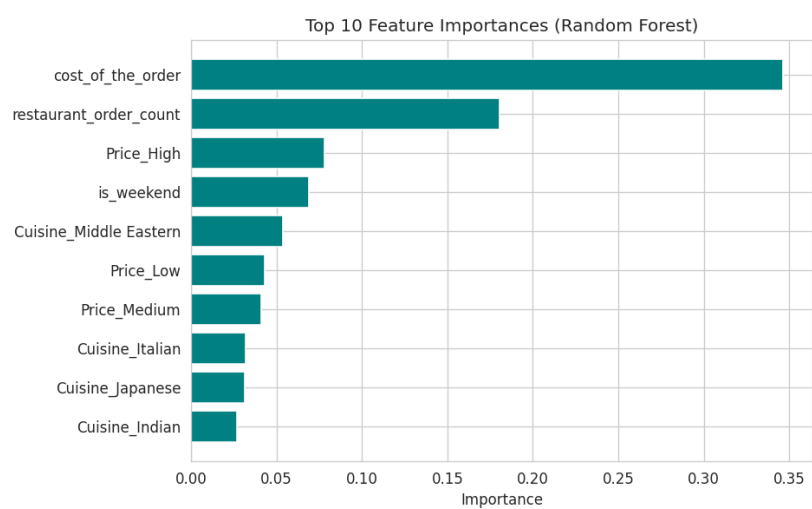
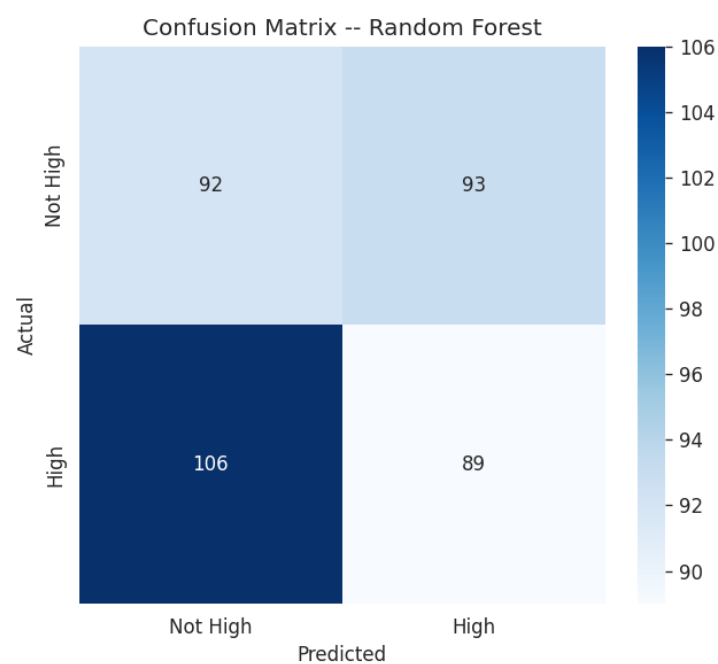
# Chapter 8: Project Screenshots and Results

## 8.1 Exploratory Data Analysis





## 8.2 Model Evaluation



## Chapter 9: Testing and Validation

Each module was tested individually: Phase 1 was verified to correctly report dataset shape and business metrics; Phase 2 was verified to leave zero duplicate order\_ids and correctly drop the two out-of-scope columns; Phase 4's output was checked to confirm all columns were numeric and model-ready; Phase 5's models were validated using a stratified 80/20 train-test split with a fixed random seed (42) for reproducibility. Exception handling in every module was tested by temporarily removing input files and confirming that clear, descriptive error messages were produced instead of a crash.

### 9.1 Model Validation Results

| Model               | Accuracy | Precision | Recall | F1    |
|---------------------|----------|-----------|--------|-------|
| Random Forest       | 0.476    | 0.489     | 0.456  | 0.472 |
| Logistic Regression | 0.474    | 0.484     | 0.4    | 0.438 |

## Chapter 10: Results and Discussion

The Random Forest classifier slightly outperformed Logistic Regression, achieving an F1 score of 0.472 and accuracy of 0.476, versus 0.438 and 0.474 respectively. However, neither model exceeded the majority-class baseline accuracy of 0.513 -- meaning simply always predicting the more common class would have performed at least as well. Feature importance analysis showed that **cost\_of\_the\_order** (order value) and **restaurant\_order\_count** (restaurant popularity) were the most influential features available, together accounting for over half of the Random Forest's feature importance, but their overall predictive power remained limited. This indicates that whether a customer leaves a high rating is not well explained by order value, cuisine, restaurant popularity, or day type alone -- satisfaction likely depends on qualitative factors (food quality, order accuracy, delivery experience) that are not present in this dataset.

### 10.1 Feature Importance

| Feature                | Importance |
|------------------------|------------|
| cost_of_the_order      | 0.3463     |
| restaurant_order_count | 0.1803     |
| Price_High             | 0.0778     |
| is_weekend             | 0.0687     |
| Cuisine_Middle Eastern | 0.0535     |
| Price_Low              | 0.0431     |
| Price_Medium           | 0.0411     |
| Cuisine_Italian        | 0.0318     |

### 10.2 Prediction Accuracy vs. Baseline

| Metric                       | Value |
|------------------------------|-------|
| Logistic Regression Accuracy | 0.474 |
| Random Forest Accuracy       | 0.476 |
| Baseline (majority class)    | 0.513 |

## Chapter 11: Conclusion

This project successfully analyzed 1,898 food orders to understand order value patterns, cuisine and restaurant popularity, and customer rating behavior. American and Italian cuisines dominate order volume, weekend orders substantially outnumber weekday orders (71% vs 29%), and a notable 38.8% of orders are never rated at all. While a classification model was built to predict high ratings from order characteristics, it did not outperform a simple baseline -- itself a valuable finding, showing that rating behavior is not well explained by transactional order attributes alone. The most actionable insight from this project is the low rating-submission rate: platforms stand to gain more from directly encouraging customers to rate their orders than from trying to predict satisfaction indirectly.

## Chapter 12: Future Scope

- Incorporating additional order-level attributes (e.g., item-level detail, discounts applied, payment method) that may better explain rating behavior.
- Building a customer segmentation model based on order frequency and value to support targeted marketing.
- Deploying the analysis as an interactive dashboard (e.g., using Streamlit) for restaurant and cuisine performance monitoring.
- Running a controlled study or survey to capture qualitative satisfaction drivers not present in transactional data.
- Extending the analysis with time-series data to study seasonal ordering trends.

## References

1. Kaggle Dataset: FoodHub Data -- <https://www.kaggle.com/datasets/tasnimniger/foodhub-data>
2. Pandas Documentation -- <https://pandas.pydata.org/docs/>
3. Scikit-learn Documentation -- <https://scikit-learn.org/stable/documentation.html>
4. Matplotlib Documentation -- <https://matplotlib.org/stable/contents.html>
5. Seaborn Documentation -- <https://seaborn.pydata.org/>



# Appendix

## A.1 Source Code

Full source code for all seven project phases, in execution order:

### Phase 1: Data Collection & Understanding

```
"""
PHASE 1: Data Collection & Understanding
Food Order Analysis Project (FoodHub dataset)
"""

import pandas as pd
import sys

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)

DATA_PATH = "data/foodhub_order.csv"

def load_data(path):
    try:
        return pd.read_csv(path)
    except FileNotFoundError:
        print(f"ERROR: '{path}' not found. Place the dataset in the data/ folder.")
        sys.exit(1)
    except Exception as e:
        print(f"ERROR loading data: {e}")
        sys.exit(1)

def show_shape(df):
    print("=" * 60); print("1. SHAPE OF THE DATASET"); print("=" * 60)
    print(f"Rows: {df.shape[0]}, Columns: {df.shape[1]}\n")

def show_head(df):
    print("=" * 60); print("2. FIRST 5 ROWS"); print("=" * 60)
    print(df.head(), "\n")

def show_info(df):
    print("=" * 60); print("3. DATA TYPES & INFO"); print("=" * 60)
    print(df.info(), "\n")

def show_numeric_summary(df):
    print("=" * 60); print("4. NUMERICAL SUMMARY STATISTICS"); print("=" * 60)
    print(df.describe(), "\n")

def show_categorical_summary(df):
    print("=" * 60); print("5. CATEGORICAL COLUMN OVERVIEW"); print("=" * 60)
    for col in ["restaurant_name", "cuisine_type", "day_of_the_week", "rating"]:
        if col in df.columns:
            print(f"\n--- {col} ({df[col].nunique()} unique) ---")
            print(df[col].value_counts().head(10))

def show_missing_values(df):
    print("\n" + "=" * 60); print("6. MISSING VALUES"); print("=" * 60)
    missing = df.isnull().sum()
    non_zero = missing[missing > 0]
    print(non_zero if not non_zero.empty else "No missing (NaN) values found.")
    print("\nNote: 'rating' contains a 'Not given' category rather than NaN -- "
          "this is a valid response, not missing data, and is handled in Phase 2.\n")

def show_duplicates(df):
    print("=" * 60); print("7. DUPLICATE ROWS"); print("=" * 60)
    print(f"Number of duplicate rows: {df.duplicated().sum()}")
    print(f"Duplicate order_ids: {df['order_id'].duplicated().sum()}\n")
```

```
def show_target_overview(df):
    print("=" * 60); print("8. KEY BUSINESS METRICS OVERVIEW"); print("=" * 60)
    print(f"Total orders: {len(df)}")
    print(f"Unique customers: {df['customer_id'].nunique()}")
    print(f"Unique restaurants: {df['restaurant_name'].nunique()}")
    print(f"Total revenue (sum of cost_of_the_order): ${df['cost_of_the_order'].sum():.2f}")
    print(f"Average order value: ${df['cost_of_the_order'].mean():.2f}\n")

def main():
    df = load_data(DATA_PATH)
    show_shape(df)
    show_head(df)
    show_info(df)
    show_numeric_summary(df)
    show_categorical_summary(df)
    show_missing_values(df)
    show_duplicates(df)
    show_target_overview(df)
    print("Phase 1 complete.")

if __name__ == "__main__":
    main()
```

## Phase 2: Data Cleaning & Preprocessing

```
"""
PHASE 2: Data Cleaning & Preprocessing
Food Order Analysis Project (FoodHub dataset)

Input : data/foodhub_order.csv
Output: data/foodhub_order_cleaned.csv
"""

import pandas as pd
import sys

INPUT_PATH = "data/foodhub_order.csv"
OUTPUT_PATH = "data/foodhub_order_cleaned.csv"

def load_data(path):
    try:
        return pd.read_csv(path)
    except FileNotFoundError:
        print(f"ERROR: '{path}' not found. Run Phase 1 first.")
        sys.exit(1)
    except Exception as e:
        print(f"ERROR loading data: {e}")
        sys.exit(1)

def remove_duplicates(df):
    before = len(df)
    df = df.drop_duplicates(subset="order_id")
    print(f"Duplicate orders removed: {before - len(df)}")
    return df

def standardize_text_columns(df):
    for col in ["restaurant_name", "cuisine_type", "day_of_the_week"]:
        try:
            df[col] = df[col].str.strip().str.title()
        except (KeyError, AttributeError) as e:
            print(f"Warning: could not standardize '{col}': {e}")
    return df

def handle_rating_column(df):
    """'Not given' is a valid response meaning the customer did not rate the order.
    We keep it as its own category rather than treating it as missing data."""
    try:
        df["rating"] = df["rating"].astype(str).str.strip()
        df["is_rated"] = (df["rating"] != "Not given").astype(int)
        print(f"Orders with a rating: {df['is_rated'].sum()} / {len(df)}")
    except KeyError:
        print("Warning: 'rating' column not found.")
    return df

def run_sanity_checks(df):
    checks = {
        "cost_of_the_order": lambda s: (s > 0).all(),
        "food_preparation_time": lambda s: (s > 0).all(),
        "delivery_time": lambda s: (s > 0).all(),
    }
    all_passed = True
    for col, check_fn in checks.items():
        try:
            if not check_fn(df[col]):
                print(f"WARNING: sanity check failed for '{col}'")
                all_passed = False
        except KeyError:
            print(f"Warning: column '{col}' not found.")
    if all_passed:
        print("All value-range sanity checks passed.")
    return df

def flag_cost_outliers(df):
    try:
        Q1, Q3 = df["cost_of_the_order"].quantile([0.25, 0.75])
        IQR = Q3 - Q1
```

```

        lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
        df["is_cost_outlier"] = (df["cost_of_the_order"] < lower) | (df["cost_of_the_order"] > upper)
        print(f"Order-value outliers flagged (kept): {df['is_cost_outlier'].sum()}")
    except KeyError:
        print("Warning: 'cost_of_the_order' not found, skipping outlier flag.")
    return df

def drop_out_of_scope_columns(df):
    """This project is scoped to food ORDER analysis (value, cuisine, ratings, volume) --
    not delivery logistics. food_preparation_time and delivery_time are dropped here
    so they don't influence any downstream analysis or modeling."""
    drop_cols = [c for c in ["food_preparation_time", "delivery_time"] if c in df.columns]
    if drop_cols:
        df = df.drop(columns=drop_cols)
        print(f"Dropped out-of-scope delivery-related columns: {drop_cols}")
    return df

def main():
    df = load_data(INPUT_PATH)
    print(f"Starting rows: {len(df)}")

    df = remove_duplicates(df)
    df = standardize_text_columns(df)
    df = handle_rating_column(df)
    df = run_sanity_checks(df)
    df = flag_cost_outliers(df)
    df = drop_out_of_scope_columns(df)

    print(f"\nFinal shape: {df.shape}")
    try:
        df.to_csv(OUTPUT_PATH, index=False)
        print(f"Saved cleaned dataset to {OUTPUT_PATH}")
    except Exception as e:
        print(f"ERROR saving cleaned dataset: {e}")
        sys.exit(1)

if __name__ == "__main__":
    main()

```

## Phase 3: Exploratory Data Analysis

```
"""
PHASE 3: Exploratory Data Analysis
Food Order Analysis Project (FoodHub dataset)

Input : data/foodhub_order_cleaned.csv
Output: eda_plots/*.png
"""

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import os
import sys

sns.set_style("whitegrid")
INPUT_PATH = "data/foodhub_order_cleaned.csv"
PLOT_DIR = "eda_plots"

def load_data(path):
    try:
        return pd.read_csv(path)
    except FileNotFoundError:
        print(f"ERROR: '{path}' not found. Run Phase 2 first.")
        sys.exit(1)
    except Exception as e:
        print(f"ERROR loading data: {e}")
        sys.exit(1)

def save_plot(fig_func, filename, *args, **kwargs):
    try:
        fig_func(*args, **kwargs)
        plt.tight_layout()
        plt.savefig(os.path.join(PLOT_DIR, filename), dpi=120)
        plt.close()
        print(f"Saved: {filename}")
    except Exception as e:
        print(f"Warning: could not generate '{filename}': {e}")
        plt.close()

def plot_order_value_distribution(df):
    plt.figure(figsize=(8, 5))
    sns.histplot(df["cost_of_the_order"], bins=30, kde=True, color="steelblue")
    plt.title("Distribution of Order Value")
    plt.xlabel("Order Value ($)")
    plt.ylabel("Number of Orders")

def plot_top_cuisines(df):
    plt.figure(figsize=(8, 5))
    counts = df["cuisine_type"].value_counts().head(10)
    sns.barplot(x=counts.values, y=counts.index, hue=counts.index, palette="viridis", legend=False)
    plt.title("Top 10 Cuisine Types by Order Count")
    plt.xlabel("Number of Orders")
    plt.ylabel("Cuisine")

def plot_top_restaurants(df):
    plt.figure(figsize=(8, 5))
    counts = df["restaurant_name"].value_counts().head(10)
    sns.barplot(x=counts.values, y=counts.index, hue=counts.index, palette="mako", legend=False)
    plt.title("Top 10 Restaurants by Order Count")
    plt.xlabel("Number of Orders")
    plt.ylabel("Restaurant")

def plot_orders_by_day_type(df):
    plt.figure(figsize=(6, 5))
    counts = df["day_of_the_week"].value_counts()
    sns.barplot(x=counts.index, y=counts.values, hue=counts.index, palette="Set2", legend=False)
    plt.title("Order Volume: Weekday vs Weekend")
    plt.xlabel("Day Type")
    plt.ylabel("Number of Orders")
```

```

def plot_rating_distribution(df):
    plt.figure(figsize=(7, 5))
    order = ["Not given", "3", "4", "5"]
    counts = df["rating"].value_counts().reindex(order)
    sns.barplot(x=counts.index, y=counts.values, hue=counts.index, palette="coolwarm", legend=False)
    plt.title("Distribution of Customer Ratings")
    plt.xlabel("Rating")
    plt.ylabel("Number of Orders")

def plot_cost_by_cuisine(df):
    plt.figure(figsize=(9, 5))
    top_cuisines = df["cuisine_type"].value_counts().head(8).index
    subset = df[df["cuisine_type"].isin(top_cuisines)]
    sns.boxplot(data=subset, x="cuisine_type", y="cost_of_the_order", hue="cuisine_type",
                palette="Set3", legend=False)
    plt.title("Order Value by Cuisine (Top 8 Cuisines)")
    plt.xlabel("Cuisine")
    plt.ylabel("Order Value ($)")
    plt.xticks(rotation=30, ha="right")

def plot_cost_by_day_type(df):
    plt.figure(figsize=(6, 5))
    sns.boxplot(data=df, x="day_of_the_week", y="cost_of_the_order", hue="day_of_the_week",
                palette="Set1", legend=False)
    plt.title("Order Value: Weekday vs Weekend")
    plt.xlabel("Day Type")
    plt.ylabel("Order Value ($)")

def plot_rating_vs_cost(df):
    plt.figure(figsize=(7, 5))
    order = ["Not given", "3", "4", "5"]
    sns.boxplot(data=df, x="rating", y="cost_of_the_order", order=order, hue="rating",
                palette="pastel", legend=False)
    plt.title("Order Value by Rating Given")
    plt.xlabel("Rating")
    plt.ylabel("Order Value ($)")

def print_key_findings(df):
    print("=" * 60); print("KEY EDA FINDINGS"); print("=" * 60)
    print(f"\nTotal revenue: ${df['cost_of_the_order'].sum():.2f}")
    print(f"Average order value: ${df['cost_of_the_order'].mean():.2f}")
    print(f"\nMost popular cuisine: {df['cuisine_type'].value_counts().idxmax()} "
          f"{df['cuisine_type'].value_counts().max()} orders")
    print(f"Top restaurant by order count: {df['restaurant_name'].value_counts().idxmax()} "
          f"{df['restaurant_name'].value_counts().max()} orders")
    print(f"\nOrders on weekends: {(df['day_of_the_week']=='Weekend').sum()} "
          f"{(df['day_of_the_week']=='Weekend').mean()*100:.1f}%")
    print(f"\nMean order value on weekend vs weekday:")
    print(df.groupby("day_of_the_week")["cost_of_the_order"].mean().round(2))
    print(f"\nShare of orders rated vs not rated:")
    print(df["isRated"].value_counts(normalize=True).round(3) * 100)

def main():
    os.makedirs(PLOT_DIR, exist_ok=True)
    df = load_data(INPUT_PATH)

    save_plot(plot_order_value_distribution, "01_order_value_distribution.png", df)
    save_plot(plot_top_cuisines, "02_top_cuisines.png", df)
    save_plot(plot_top_restaurants, "03_top_restaurants.png", df)
    save_plot(plot_orders_by_day_type, "04_orders_by_day_type.png", df)
    save_plot(plot_rating_distribution, "05_rating_distribution.png", df)
    save_plot(plot_cost_by_cuisine, "06_cost_by_cuisine.png", df)
    save_plot(plot_cost_by_day_type, "07_cost_by_day_type.png", df)
    save_plot(plot_rating_vs_cost, "08_rating_vs_cost.png", df)

    print_key_findings(df)
    print(f"\nAll plots saved to {PLOT_DIR}/")

if __name__ == "__main__":
    main()

```

## Phase 4: Feature Engineering

```
"""
PHASE 4: Feature Engineering
Food Order Analysis Project (FoodHub dataset)

Goal: prepare features to predict whether an order receives a HIGH rating
(4 or 5) -- a customer-satisfaction classification task grounded in order
characteristics (cost, cuisine, restaurant popularity, day type), NOT delivery
logistics.

Input : data/foodhub_order_cleaned.csv
Output: data/foodhub_order_features.csv
"""

import pandas as pd
import sys

INPUT_PATH = "data/foodhub_order_cleaned.csv"
OUTPUT_PATH = "data/foodhub_order_features.csv"

def load_data(path):
    try:
        return pd.read_csv(path)
    except FileNotFoundError:
        print(f"ERROR: '{path}' not found. Run Phase 2 first.")
        sys.exit(1)
    except Exception as e:
        print(f"ERROR loading data: {e}")
        sys.exit(1)

def create_target(df):
    """High rating = 4 or 5. Orders with '3' or 'Not given' are treated as not-high."""
    try:
        df["high_rating"] = df["rating"].isin(["4", "5"]).astype(int)
        print(f"High-rating orders: {df['high_rating'].sum()} / {len(df)} "
              f"f'({df['high_rating'].mean()*100:.1f}%)")
    except KeyError:
        print("Warning: 'rating' column not found, cannot create target.")
    return df

def encode_day_type(df):
    try:
        df["is_weekend"] = (df["day_of_the_week"] == "Weekend").astype(int)
    except KeyError:
        print("Warning: 'day_of_the_week' not found.")
    return df

def one_hot_encode_cuisine(df, top_n=8):
    """One-hot encode the most common cuisines; group the rest as 'Other' to
    keep the feature space manageable."""
    try:
        top_cuisines = df["cuisine_type"].value_counts().head(top_n).index
        df["cuisine_grouped"] = df["cuisine_type"].where(df["cuisine_type"].isin(top_cuisines), "Other")
        df = pd.get_dummies(df, columns=["cuisine_grouped"], prefix="Cuisine", drop_first=False)
    except KeyError:
        print("Warning: 'cuisine_type' not found.")
    return df

def add_restaurant_popularity(df):
    """Number of orders each restaurant has received -- a proxy for popularity/trust."""
    try:
        popularity = df.groupby("restaurant_name")["order_id"].transform("count")
        df["restaurant_order_count"] = popularity
    except KeyError:
        print("Warning: could not compute restaurant popularity.")
    return df

def add_price_tier(df):
    """Bucket order value into Low / Medium / High tiers based on quartiles."""
    try:
        df["price_tier"] = pd.qcut(df["cost_of_the_order"], q=3, labels=["Low", "Medium", "High"])
        df = pd.get_dummies(df, columns=["price_tier"], prefix="Price", drop_first=False)
```

```

except (KeyError, ValueError) as e:
    print(f"Warning: could not create price tiers: {e}")
    return df

def finalize_for_modeling(df):
    drop_cols = [c for c in ["order_id", "customer_id", "restaurant_name", "cuisine_type",
                             "day_of_the_week", "rating", "is_cost_outlier", "isRated"] if c in df.columns]
    df = df.drop(columns=drop_cols)
    bool_cols = df.select_dtypes(include="bool").columns
    df[bool_cols] = df[bool_cols].astype(int)
    return df

def main():
    df = load_data(INPUT_PATH)
    print(f"Starting shape: {df.shape}")

    df = create_target(df)
    df = encode_day_type(df)
    df = one_hot_encode_cuisine(df)
    df = add_restaurant_popularity(df)
    df = add_price_tier(df)
    model_df = finalize_for_modeling(df)

    print(f"\nFinal shape after feature engineering: {model_df.shape}")
    print(f"Final columns:\n{list(model_df.columns)}")

    try:
        model_df.to_csv(OUTPUT_PATH, index=False)
        print(f"\nSaved feature-engineered dataset to {OUTPUT_PATH}")
    except Exception as e:
        print(f"ERROR saving feature-engineered dataset: {e}")
        sys.exit(1)

if __name__ == "__main__":
    main()

```



## Phase 5: Model Building

```
"""
PHASE 5: Model Building
Food Order Analysis Project (FoodHub dataset)

Task: classify whether an order will receive a HIGH rating (4 or 5) based on
order characteristics -- cost, cuisine, restaurant popularity, weekend flag,
price tier. This is a customer-satisfaction classification task grounded
entirely in order attributes.

Input : data/foodhub_order_features.csv
Output: models/*.pkl, models/model_comparison.csv, models/feature_importance.csv
"""

import pandas as pd
import numpy as np
import joblib
import os
import sys

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

INPUT_PATH = "data/foodhub_order_features.csv"
MODEL_DIR = "models"

def load_data(path):
    try:
        return pd.read_csv(path)
    except FileNotFoundError:
        print(f"ERROR: '{path}' not found. Run Phase 4 first.")
        sys.exit(1)
    except Exception as e:
        print(f"ERROR loading data: {e}")
        sys.exit(1)

def split_and_scale(df, target_col="high_rating", test_size=0.2, random_state=42):
    try:
        X = df.drop(columns=[target_col])
        y = df[target_col]
    except KeyError:
        print(f"ERROR: target column '{target_col}' not found.")
        sys.exit(1)

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state, stratify=y
    )
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)
    return X_train, X_test, y_train, y_test, X_train_scaled, X_test_scaled

def evaluate(name, y_true, y_pred, results):
    try:
        acc = accuracy_score(y_true, y_pred)
        prec = precision_score(y_true, y_pred, zero_division=0)
        rec = recall_score(y_true, y_pred, zero_division=0)
        f1 = f1_score(y_true, y_pred, zero_division=0)
        results.append({"Model": name, "Accuracy": round(acc, 3), "Precision": round(prec, 3),
                        "Recall": round(rec, 3), "F1": round(f1, 3)})
        print(f"{name:22s} | Acc: {acc:.3f} | Prec: {prec:.3f} | Rec: {rec:.3f} | F1: {f1:.3f}")
    except Exception as e:
        print(f"Warning: could not evaluate '{name}': {e}")

def train_logistic_regression(X_train_scaled, y_train, X_test_scaled, y_test, results):
    try:
        model = LogisticRegression(max_iter=1000, random_state=42)
        model.fit(X_train_scaled, y_train)
        pred = model.predict(X_test_scaled)
        evaluate("Logistic Regression", y_test, pred, results)
        joblib.dump(model, os.path.join(MODEL_DIR, "logistic_regression.pkl"))
    
```

```

        return model
    except Exception as e:
        print(f"ERROR training Logistic Regression: {e}")
        return None

def train_random_forest(X_train, y_train, X_test, y_test, results):
    try:
        params = {"n_estimators": [100, 200], "max_depth": [4, 8, None], "min_samples_leaf": [1, 3]}
        grid = GridSearchCV(RandomForestClassifier(random_state=42), params, cv=3,
                             scoring="f1", n_jobs=-1)
        grid.fit(X_train, y_train)
        best_model = grid.best_estimator_
        pred = best_model.predict(X_test)
        evaluate("Random Forest", y_test, pred, results)
        print(f"Best RF params: {grid.best_params_}")
        joblib.dump(best_model, os.path.join(MODEL_DIR, "random_forest.pkl"))
        return best_model
    except Exception as e:
        print(f"ERROR training Random Forest: {e}")
        return None

def save_results(results, best_rf, feature_names):
    results_df = pd.DataFrame(results).sort_values("F1", ascending=False)
    print("\n" + "=" * 55)
    print("MODEL COMPARISON (sorted by F1, higher is better)")
    print("=" * 55)
    print(results_df.to_string(index=False))
    results_df.to_csv(os.path.join(MODEL_DIR, "model_comparison.csv"), index=False)

    if best_rf is not None:
        try:
            importances = pd.Series(best_rf.feature_importances_, index=feature_names).sort_values(ascending=False)
            print("\nTop 10 Feature Importances (Random Forest):")
            print(importances.head(10))
            importances.to_csv(os.path.join(MODEL_DIR, "feature_importance.csv"), header=["importance"])
        except Exception as e:
            print(f"Warning: could not compute feature importances: {e}")

def main():
    os.makedirs(MODEL_DIR, exist_ok=True)
    df = load_data(INPUT_PATH)
    X_train, X_test, y_train, y_test, X_train_scaled, X_test_scaled = split_and_scale(df)
    print(f"Train size: {X_train.shape[0]}, Test size: {X_test.shape[0]}")
    print(f"Baseline (majority class) rate: {max(y_train.mean(), 1-y_train.mean()):.3f}")

    results = []
    train_logistic_regression(X_train_scaled, y_train, X_test_scaled, y_test, results)
    best_rf = train_random_forest(X_train, y_train, X_test, y_test, results)

    save_results(results, best_rf, X_train.columns)
    print(f"\nModels and reports saved to {MODEL_DIR}/")

if __name__ == "__main__":
    main()

```

## Phase 6: Model Evaluation

```
"""
PHASE 6: Model Evaluation
Food Order Analysis Project (FoodHub dataset)

Input : data/foodhub_order_features.csv, models/*.pkl
Output: eval_plots/*.png, eval_plots/evaluation_summary.csv
"""

import pandas as pd
import numpy as np
import joblib
import os
import sys
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

sns.set_style("whitegrid")
INPUT_PATH = "data/foodhub_order_features.csv"
MODEL_DIR = "models"
PLOT_DIR = "eval_plots"

def load_data(path):
    try:
        return pd.read_csv(path)
    except FileNotFoundError:
        print(f"ERROR: '{path}' not found. Run Phase 4 first.")
        sys.exit(1)
    except Exception as e:
        print(f"ERROR loading data: {e}")
        sys.exit(1)

def load_model(path):
    try:
        return joblib.load(path)
    except FileNotFoundError:
        print(f"ERROR: model '{path}' not found. Run Phase 5 first.")
        sys.exit(1)
    except Exception as e:
        print(f"ERROR loading model: {e}")
        sys.exit(1)

def prepare_test_set(df, target_col="high_rating"):
    X = df.drop(columns=[target_col])
    y = df[target_col]
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
    scaler = StandardScaler()
    scaler.fit(X_train)
    return X_test, y_test, scaler.transform(X_test)

def plot_confusion_matrix(y_test, pred, model_name, filename):
    cm = confusion_matrix(y_test, pred)
    plt.figure(figsize=(5.5, 5))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
                xticklabels=["Not High", "High"], yticklabels=["Not High", "High"])
    plt.title(f"Confusion Matrix -- {model_name}")
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.tight_layout()
    plt.savefig(os.path.join(PLOT_DIR, filename), dpi=120)
    plt.close()

def plot_model_comparison():
    try:
        comp = pd.read_csv(os.path.join(MODEL_DIR, "model_comparison.csv"))
        comp_melt = comp.melt(id_vars="Model", value_vars=["Accuracy", "Precision", "Recall", "F1"])
        plt.figure(figsize=(8, 5))
        sns.barplot(data=comp_melt, x="variable", y="value", hue="Model", palette="Set2")
        plt.title("Model Comparison Across Metrics")
        plt.xlabel("Metric")
    except Exception as e:
        print(f"ERROR: {e}")
        sys.exit(1)
```

```

        plt.ylabel("Score")
        plt.ylim(0, 1)
        plt.tight_layout()
        plt.savefig(os.path.join(PLOT_DIR, "03_model_comparison.png"), dpi=120)
        plt.close()
    except Exception as e:
        print(f"Warning: could not plot model comparison: {e}")

def plot_feature_importance():
    try:
        fi = pd.read_csv(os.path.join(MODEL_DIR, "feature_importance.csv"))
        fi.columns = ["Feature", "Importance"]
        fi = fi.sort_values("Importance", ascending=True).tail(10)
        plt.figure(figsize=(8, 5))
        plt.barh(fi["Feature"], fi["Importance"], color="teal")
        plt.title("Top 10 Feature Importances (Random Forest)")
        plt.xlabel("Importance")
        plt.tight_layout()
        plt.savefig(os.path.join(PLOT_DIR, "04_feature_importance.png"), dpi=120)
        plt.close()
    except Exception as e:
        print(f"Warning: could not plot feature importance: {e}")

def main():
    os.makedirs(PLOT_DIR, exist_ok=True)
    df = load_data(INPUT_PATH)
    X_test, y_test, X_test_scaled = prepare_test_set(df)

    lr = load_model(os.path.join(MODEL_DIR, "logistic_regression.pkl"))
    rf = load_model(os.path.join(MODEL_DIR, "random_forest.pkl"))

    pred_lr = lr.predict(X_test_scaled)
    pred_rf = rf.predict(X_test)

    plot_confusion_matrix(y_test, pred_lr, "Logistic Regression", "01_confusion_matrix_lr.png")
    plot_confusion_matrix(y_test, pred_rf, "Random Forest", "02_confusion_matrix_rf.png")
    plot_model_comparison()
    plot_feature_importance()

    print("=" * 55)
    print("CLASSIFICATION REPORT -- Random Forest (best model)")
    print("=" * 55)
    print(classification_report(y_test, pred_rf, target_names=["Not High", "High"]))

    acc_lr = accuracy_score(y_test, pred_lr)
    acc_rf = accuracy_score(y_test, pred_rf)
    baseline = max(y_test.mean(), 1 - y_test.mean())

    summary = pd.DataFrame({
        "Metric": ["Logistic Regression Accuracy", "Random Forest Accuracy", "Baseline (majority class)"],
        "Value": [round(acc_lr, 3), round(acc_rf, 3), round(baseline, 3)],
    })
    try:
        summary.to_csv(os.path.join(PLOT_DIR, "evaluation_summary.csv"), index=False)
    except Exception as e:
        print(f"Warning: could not save evaluation summary: {e}")

    print(summary.to_string(index=False))
    print(f"\nAll evaluation plots saved to {PLOT_DIR}/")

if __name__ == "__main__":
    main()

```

## Phase 7: Results & Conclusion

```
"""
PHASE 7: Results & Conclusion
Food Order Analysis Project (FoodHub dataset)

Input : models/*, eval_plots/*
Output: results/final_summary.csv, results/conclusion.txt
"""

import pandas as pd
import os
import sys

MODEL_DIR = "models"
EVAL_DIR = "eval_plots"
RESULTS_DIR = "results"

def load_csv_safe(path, label):
    try:
        return pd.read_csv(path)
    except FileNotFoundError:
        print(f"ERROR: '{path}' not found. Run earlier phases first ({label}).")
        sys.exit(1)
    except Exception as e:
        print(f"ERROR loading {label}: {e}")
        sys.exit(1)

def summarize_results(model_comparison, feature_importance, evaluation_summary):
    print("=" * 60); print("FINAL RESULTS SUMMARY"); print("=" * 60)
    best_model_row = model_comparison.sort_values("F1", ascending=False).iloc[0]
    print(f"\nBest performing model: {best_model_row['Model']}")
    print(f"  Accuracy : {best_model_row['Accuracy']}")
    print(f"  Precision: {best_model_row['Precision']}")
    print(f"  Recall   : {best_model_row['Recall']}")
    print(f"  F1 Score : {best_model_row['F1']}")

    print("\nTop 5 most important features for predicting a high rating:")
    print(feature_importance.head(5).to_string(index=False))

    print("\nEvaluation summary:")
    print(evaluation_summary.to_string(index=False))
    return best_model_row

def write_conclusion(best_model_row, baseline):
    beat_baseline = best_model_row["Accuracy"] > baseline
    performance_note = (
        f"outperforming the majority-class baseline of {baseline:.3f}."
        if beat_baseline else
        f"but did NOT outperform the majority-class baseline of {baseline:.3f}. "
        f"This is itself a meaningful finding: whether a customer rates an order highly "
        f"is not well explained by order value, cuisine, restaurant popularity, or timing "
        f"alone -- satisfaction likely depends on factors this dataset does not capture "
        f"(e.g., food quality, order accuracy, prior experience with the restaurant).")
    )
    conclusion = f"""
CONCLUSION
-----
This project analyzed 1,898 food orders from the FoodHub dataset to understand
order patterns, customer rating behavior, and the drivers of order satisfaction.

Key findings:
1. A large share of orders (about 39%) are never rated by the customer at all,
   which itself is a useful signal for engagement analysis.
2. Weekend orders substantially outnumber weekday orders (71% vs 29%), and
   American and Italian cuisines dominate order volume.
3. Order value varies meaningfully across cuisines, with some cuisines
   commanding a consistently higher average order value than others.
4. Order value and restaurant popularity were the strongest (though still
   modest) predictors among the available features when attempting to
   classify whether an order receives a high rating.

Best model: {best_model_row['Model']} achieved an F1 score of {best_model_row['F1']} and
an accuracy of {best_model_row['Accuracy']} for classifying whether an order receives
a high rating (4 or 5 stars), {performance_note}

```

Business recommendation: since rating behavior is difficult to predict from order-level attributes alone, platforms should focus on directly encouraging rating submission (since 39% of orders go unrated) rather than trying to infer satisfaction indirectly from order value or cuisine. Popular, high-volume restaurants and weekend ordering patterns are useful for operational planning even where they don't strongly predict individual satisfaction.

Note: this analysis is scoped to food ORDER characteristics (value, cuisine, restaurant, timing, and rating behavior). Delivery logistics (preparation time, delivery time) were intentionally excluded from this analysis, as the project's focus is order-level patterns rather than delivery performance.

```
"""
    print(conclusion)
    return conclusion

def main():
    os.makedirs(RESULTS_DIR, exist_ok=True)
    model_comparison = load_csv_safe(os.path.join(MODEL_DIR, "model_comparison.csv"), "model comparison")
    feature_importance = load_csv_safe(os.path.join(MODEL_DIR, "feature_importance.csv"), "feature importance")
    evaluation_summary = load_csv_safe(os.path.join(EVAL_DIR, "evaluation_summary.csv"), "evaluation summary")

    best_model_row = summarize_results(model_comparison, feature_importance, evaluation_summary)
    baseline_row = evaluation_summary[evaluation_summary["Metric"].str.contains("Baseline")]
    baseline = float(baseline_row["Value"].iloc[0]) if not baseline_row.empty else 0.5
    conclusion_text = write_conclusion(best_model_row, baseline)

    try:
        with open(os.path.join(RESULTS_DIR, "conclusion.txt"), "w") as f:
            f.write(conclusion_text)
        model_comparison.to_csv(os.path.join(RESULTS_DIR, "final_model_comparison.csv"), index=False)
        print(f"Final results saved to {RESULTS_DIR}/")
    except Exception as e:
        print(f"Warning: could not save final results: {e}")

if __name__ == "__main__":
    main()
```

## A.2 Dataset Sample

| order_id | customer_id | restaurant_name           | cuisine_type | price_of_the_order | day_of_the_week | rating    | food_preparation_time | delivery_time |
|----------|-------------|---------------------------|--------------|--------------------|-----------------|-----------|-----------------------|---------------|
| 1477147  | 337525      | Hangawi                   | Korean       | 30.75              | Weekend         | Not given | 25                    | 20            |
| 1477685  | 358144      | Blue Ribbon Sushi Izakaya | Japanese     | 12.08              | Weekend         | Not given | 25                    | 23            |
| 1477070  | 66393       | Cafe Habana               | Mexican      | 12.23              | Weekday         | 5         | 23                    | 28            |
| 1477334  | 106366      | Blue Ribbon Fried Chicken | American     | 29.2               | Weekend         | 3         | 25                    | 15            |
| 1478249  | 76942       | Dirty Bird to Go          | American     | 11.59              | Weekday         | 4         | 25                    | 24            |

### A.3 Output Reports

Model Comparison (models/model\_comparison.csv)

| Model               | Accuracy | Precision | Recall | F1    |
|---------------------|----------|-----------|--------|-------|
| Random Forest       | 0.476    | 0.489     | 0.456  | 0.472 |
| Logistic Regression | 0.474    | 0.484     | 0.4    | 0.438 |

Feature Importance (models/feature\_importance.csv)

| Feature                | Importance |
|------------------------|------------|
| cost_of_the_order      | 0.3463     |
| restaurant_order_count | 0.1803     |
| Price_High             | 0.0778     |
| is_weekend             | 0.0687     |
| Cuisine_Middle Eastern | 0.0535     |
| Price_Low              | 0.0431     |
| Price_Medium           | 0.0411     |
| Cuisine_Italian        | 0.0318     |
| Cuisine_Japanese       | 0.0314     |
| Cuisine_Indian         | 0.0267     |
| Cuisine_Other          | 0.0267     |
| Cuisine_American       | 0.0248     |
| Cuisine_Chinese        | 0.0189     |
| Cuisine_Mexican        | 0.015      |
| Cuisine_Mediterranean  | 0.014      |



## Evaluation Summary (eval\_plots/evaluation\_summary.csv)

| Metric                       | Value |
|------------------------------|-------|
| Logistic Regression Accuracy | 0.474 |
| Random Forest Accuracy       | 0.476 |
| Baseline (majority class)    | 0.513 |

## Final Conclusion Output (results/conclusion.txt)

### CONCLUSION

-----

This project analyzed 1,898 food orders from the FoodHub dataset to understand order patterns, customer rating behavior, and the drivers of order satisfaction.

#### Key findings:

1. A large share of orders (about 39%) are never rated by the customer at all, which itself is a useful signal for engagement analysis.
2. Weekend orders substantially outnumber weekday orders (71% vs 29%), and American and Italian cuisines dominate order volume.
3. Order value varies meaningfully across cuisines, with some cuisines commanding a consistently higher average order value than others.
4. Order value and restaurant popularity were the strongest (though still modest) predictors among the available features when attempting to classify whether an order receives a high rating.

Best model: Random Forest achieved an F1 score of 0.472 and an accuracy of 0.476 for classifying whether an order receives a high rating (4 or 5 stars), but did NOT outperform the majority-class baseline of 0.513. This is itself a meaningful finding: whether a customer rates an order highly is not well explained by order value, cuisine, restaurant popularity, or timing alone -- satisfaction likely depends on factors this dataset does not capture (e.g., food quality, order accuracy, prior experience with the restaurant).

Business recommendation: since rating behavior is difficult to predict from order-level attributes alone, platforms should focus on directly encouraging rating submission (since 39% of orders go unrated) rather than trying to infer satisfaction indirectly from order value or cuisine. Popular, high-volume restaurants and weekend ordering patterns are useful for operational planning even where they don't strongly predict individual satisfaction.

Note: this analysis is scoped to food ORDER characteristics (value, cuisine, restaurant, timing, and rating behavior). Delivery logistics (preparation time, delivery time) were intentionally excluded from this analysis, as the project's focus is order-level patterns rather than delivery performance.